

Learnings & Reflections Document

1. Team Reflection

- How did the team work together?
 - We made sure that each team member worked on every aspect of the project, so both of us contributed to the code, circuit and build. We either built on top of each other's work (bases to final), or worked parallelly. For example, while one member worked on code, the other was working on the model.
- What worked well?
 - Creating shared checklists (which improved accountability), building simultaneously (eg: debugging each other's code, double-checking each other's circuitry) and starting the preliminary coding and circuitry early (helped us develop our idea better).
 - Constant communication and idea-sharing helped us develop and improve our ideas while allowing for quick trouble-shooting.
- What could be improved?
 - Creating small-scale prototypes with the materials we were going to use (rather than just with cardboard) would have helped us "fail-fast" and understand the real material properties better.
 - Leaving more time for the final build. Our main focus was the circuitry and coding, especially since we were working with a new component (MPU 6050) and trying to understand how to connect its movements to the webpage properly.

2. Technical Reflection

- Key technical challenges
 - The MPU 6050 involved multiple iterative codes due to jittery and drifting movement. Since we were also using Generative AI for this (as we had not used the sensor before this project), we had to understand the code line by line first, so that we could trouble-shoot effectively. We researched, ran and tested codes from multiple sources (GitHub, YouTube, Claude, etc) until we managed to make the gyroscope and pointer move smoothly.
 - Our NeoPixel ended up getting shorted on the day of the exhibition, despite correct connections on the power supply module. The system was running smoothly (LDR

was detected and we knew because of print statements) but light output stopped. This was an internal failure from a component we had sourced from outside.

- On the build side- we struggled with the lightsaber's tube. When we initially started, we were recommended 3mm acrylic to bend but once we made that tube, it became too heavy to hold up. The 3-D printing material broke apart quickly and so we tried to laser cut instead. Eventually, we ended up using cardboard as that was the easiest to keep and remove our circuit in (especially since the circuit had parts that needed to be connected outside the breadboard).
- What did you learn (hardware/software)?
 - Learnt how to make the most out of a small breadboard (using only one side of the ESP32 so the hilt is not too wide). Our hardware turned out to be an unexpected constraint for us.
 - Learnt new code because of our MPU 6050. We had to install a new library called "mpu6050". Using this and the "SoftI2C" module, we created a code for printing the threshold values of the gyroscope axes. This gave us a better understanding of what our MPU 6050 did. We used the axes as a reference for the pointer onscreen and learned about how the ESP32 communicates between the physical gyroscope and digital pointer (through JSON).
 - Learned how to consider the circuit's size, shape and orientation into the build- where we wanted the LDR and push button positioned, how the NeoPixel would sit within our lightsaber.
- Mistakes & debugging experiences
 - Had to test out different Loop sequences for the LDR -> NeoPixel -> Push Button -> LDR sequence. We wanted the lightsaber to be intuitively activated (only when LDR was high).
 - Multiple buggy codes of our gesture-controller to video game connection- where the pointer either didn't move in certain zones, axes were inverted or the pointer lagged. We started with the basics (axes thresholds) and had to experiment a lot to get the final code (both with AI and by learning the code).
 - Combining all our individual codes into one also involved iterations to see how the loops would run and prevent any blocking (especially as the video game code was heavy and we wanted all codes to run simultaneously).

3. Design Reflection

- Did the experience match your intention?

- Our process was very iterative and involved a lot of back-and-forth which matched the expectations of any design process. We faced many mechanical problems while the code and circuitry part went relatively smoothly. Overall, our final outcome is very different from what we initially came up with, and we did the best after each obstacle with the knowledge and resources we had (especially since our build broke on the day of). Our intent was to make something simulative and fun, which we believe we achieved by through our hybrid video game.
- What would you improve?
 - More “fail-fast” prototypes. One of our major challenges was materials and how they affected the build of our project. We realized many things while working on the build, after all the code and circuitry had been completed.
- User feedback insights
 - Users were very excited about the light activation
 - They were excited to hear that there was a simulative game that could be played
 - Users had difficulty with the hilt and pointer movement matching. Since the gyroscope detected wrist angle and tilt, we had to remind them to rotate their wrist in the chosen direction rather than their whole arm/the light saber.

4. Personal Reflection

Emma:

- What I Learned

This project introduced me to components I had never worked with before, especially the MPU. I learned how it works, how to calibrate it, and how to actually get useful data out of it. It also taught me how to combine different pieces of code from different sources and debug them until everything worked together as one system.

- What I Struggled With

The physical build gave us a lot of trouble. Putting everything together in a way that was stable and functional was harder than expected, and we ran into multiple failures along the way that forced us to rethink and redo parts of the build, including some last-minute fixes right before the deadline.

- What I Am Proud Of

I'm proud that we managed to combine Java, Python, and HTML code into one working project. On top of that, calibrating the MPU and getting it to work the way we needed it to felt like a real accomplishment. Despite all the setbacks, we pulled it together in the end.

Aanya:

- What I learned:

Technically, I learned new code, how to use new components (the MPU 6050), and how to make the hardware and software components all work together. This process made me understand each new code line by line and make manual changes to fix bugs, despite the code being AI generated initially. Prompting AI to fix the bugs for some complicated parts also involved understanding the base code properly myself. Overall, I learned how to pivot when things fail (like the lightsaber tube) and have patience with iterations (when trying to match the MPU with the game).

- What I struggled with:

The MPU 6050 data and pointer movement code was very buggy and involved lots of iterations. Our build gave us a lot of trouble, down to the final day. We tried multiple solutions (from sourcing different acrylic sheets to cellophane) and ended up with a last-minute fix. We also ran into many unexpected circuitry issues- like the LDR's sensitivity randomly changing, which required constant trouble-shooting.

- What I am proud of:

I am proud that our code worked, especially because it involved combining many different codes, some of which was new to me. I am proud that we could still show a functioning model to play the game with, even if it wasn't exactly what we envisioned. Despite many unexpected challenges, we managed to present a functioning system.